📊 Final Group Project Technical Report

AuraCart Retail Analytics Predictive Engine

Module: ITS 2140: Machine Learning: Foundations to Production Systems
Date: April 2026
Group: Group 13

👥 Group Members

Student Name	Registration Number
Rashmika Navod	`2301691001`
Lakmal kumarasiri	`2301691034`
Chamath dilshan	`2301691093`
Sachini poornima	`2301691010`
Sandunil Malik Bandara	`2301691016`

📑 Table of Contents

- 1. Executive Summary
- 2. Introduction and Business Context
  - 2.1 Project Overview
  - 2.2 Business Problem at AuraCart
  - 2.3 Project Objectives
  - 2.4 Overview of the Dataset
- 3. Data Exploration and Preprocessing
  - 3.1 Dataset Overview
  - 3.2 Exploratory Data Analysis (EDA)
  - 3.3 Analysis of Continuous Variables
  - 3.4 Categorical Variable Analysis
  - 3.5 Correlation Analysis
  - 3.6 Key Findings from EDA
  - 3.7 Data Preprocessing Pipeline
  - 3.8 Handling Missing Values
  - 3.9 Categorical Encoding
  - 3.10 Feature Scaling
  - 3.11 Addressing Class Imbalance
- 4. Regression and Classification Modeling
  - 4.1 Continuous Price Prediction using Multiple Linear Regression
  - 4.2 Regression Training Process and Hyperparameter Tuning
  - 4.3 Multi-class Classification using Softmax Regression
  - 4.4 Classification Training Process
  - 4.5 Decision Threshold Adjustment
  - 4.6 MLflow Experiment Tracking

- 5. Model Evaluation and Performance Analysis
  - 5.1 Regression Evaluation using MSE and MAE
  - 5.2 Cross-Validation Analysis
  - 5.3 Underfitting and Overfitting Discussion
  - 5.4 Classification Evaluation
  - 5.5 Confusion Matrix Analysis
  - 5.6 Precision, Recall, and F1-Score
  - 5.7 Precision–Recall Trade-offs
  - 5.8 Business Risk and Asymmetric Error Analysis
- 6. Customer Behavior Clustering
  - 6.1 Introduction to Unsupervised Learning
  - 6.2 K-Means Clustering Approach
  - 6.3 Selecting the Number of Clusters
  - 6.4 Elbow Method and Silhouette Score
  - 6.5 Cluster Interpretation
  - 6.6 Business Insights from Clusters
- 7. Production Deployment and MLOps Workflow
  - 7.1 MLflow Experiment Tracking
  - 7.2 Final Model Selection
  - 7.3 Unified Prediction Pipeline
  - 7.4 Model Serialization using Joblib
  - 7.5 Requirements and Dependency Management
  - 7.6 Uploading Artifacts to Google Cloud Storage
  - 7.7 Vertex AI Model Deployment
  - 7.8 Live Endpoint Testing
  - 7.9 Successful Prediction Response Evidence
- 8. Conclusion and Future Work
- 9. References
- 10. Appendices
  - 10.1 Screenshots of MLflow Runs
  - 10.2 Vertex AI Deployment Screenshots
  - 10.3 Sample JSON Prediction Request and Response
  - 10.4 Additional Graphs and Visualizations
  - 10.5 Code Snippets / Notebook Structure

🚀 1. Executive Summary

> *[!IMPORTANT] AuraCart Retail Analytics, a rapidly scaling digital e-commerce platform, recently identified a critical necessity to transition away from intuition-based decision-making methodologies toward a highly automated, data-centric predictive system. Facing unprecedented transaction volumes and highly volatile supply chain logistics, the organization's executive leadership mandated the design and deployment of a multi-faceted machine learning architecture capable of dynamic revenue forecasting, customer segmentation, logistical bottleneck prediction, and targeted behavioral clustering.*

This technical report delineates the end-to-end development life-cycle of the intelligent predictive analytics engine commissioned by AuraCart. To accomplish the mandate, an external Machine Learning Engineering team meticulously analyzed the "E-commerce Customer Order Behavior Dataset", encompassing 10,000 distinct transactional records. Our team engineered a unified solution comprising three concurrent algorithmic pillars: Continuous Regression for

pricing predictions, Multi-class Classification for customer segment and delivery status forecasting, and Unsupervised Clustering to unveil hidden buyer behaviors.

Our findings yielded highly actionable business intelligence. We identified that 'Returned' orders could be dynamically predicted and flagged using Softmax Regression integrated with calibrated class-weighting penalties to circumvent the dataset's inherent class imbalance. Concurrently, an integrated K-Means algorithmic pipeline unveiled complex latency disparities, cleanly segmenting customers by their shipping delay experiences and purchasing volumes, thereby allowing for hyper-targeted marketing and retention interventions.

Beyond isolated offline model metrics, the ultimate success of the project rested upon seamless productionization. To achieve this, the entire experimental workflow was rigorously cataloged using the MLflow software ecosystem. The finalized champion model was successfully serialized into a standardized `model.joblib` format, migrated to a Google Cloud Storage environment, and formally deployed as a scalable RESTful inference endpoint via Google Cloud Vertex AI infrastructure.

🏢 2. Introduction and Business Context

2.1 Project Overview

Modern digital commerce operates on razor-thin margins heavily dictated by logistical efficiency and highly personalized user experiences. As user demographics grow increasingly heterogeneous, AuraCart Retail Analytics sought a technological shift. Thus, the foundation of this project is the conception, mathematical optimization, and cloud deployment of an intelligent forecasting system that uses machine learning to generate actionable inferences.

2.2 Business Problem at AuraCart

Currently operating as a mid-tier digital retail platform, AuraCart has observed severe operational friction explicitly resulting from manual heuristics and lagging historical reporting mechanisms. Relying on generalized logic to predict delivery failure, assess product pricing elasticity, and target promotional materials translates rapidly to compounding financial losses and delayed strategic adaptations.

2.3 Project Objectives

The executive mandate authorized for this project solves these shortfalls by targeting three algorithmic verticals:

1. **Dynamic Forecasting (Regression):** Predict the final monetary value of incoming unpriced transactions to enable dynamic forecasting and optimize inventory liquidity.
2. **Logistical Risk Mitigation (Classification):** Categorize unstructured transactions into proactive delivery status outcomes (Delivered, Shipped, Pending, Returned) to act as an early-warning mechanism and categorize buyer segments.
3. **Behavioral Target Clustering (Unsupervised Learning):** Process incoming numerical features through unsupervised clustering to find latent similarities unidentifiable by human supervisors, generating profiles to drive precision-marketing.

2.4 Overview of the Dataset

This project relies exclusively upon the `millat/e-commerce-orders` dataset sourced from the Hugging Face hub, featuring 10,000 granular commercial records. This dataset captures the inherent messiness and heterogeneous class distribution reflective of real-world operational realities, necessitating robust data-engineering tactics. Features include quantitative details like item quantity, and price, and categorical features such as payment method and user device type.

📊 3. Data Exploration and Preprocessing

3.1 Dataset Overview

The foundational data encompasses 10,000 distinct transactional records spanning diverse customer interactions. The preliminary load process utilized Python's core `pandas` library, revealing distinct continuous numerical, categorical grouped, and datetime oriented variables detailing purchases. Arbitrary distinct identifiers like `order_id` and `customer_id` were dropped natively.

3.2 Exploratory Data Analysis (EDA)

A rigorous Exploratory Data Analysis (EDA) framework was executed utilizing `matplotlib` and `seaborn`. The analytical scope focused on extracting statistical characteristics, inspecting distributions, and mapping feature intersections. EDA guided the structural methodology defining exactly how the analytical data pipeline would operate.

3.3 Analysis of Continuous Variables

The primary continuous target variable, `price`, exhibited a multi-modal distribution ranging from $5 to $500, with dense clusters operating largely at nominal value thresholds. Features such as `Quantity` operated optimally as discrete numericals, predominantly spread uniformly between integers 1 and 10. Temporal properties mapped from raw `order_date` metrics extracted specific behavioral numeric metadata such as `order_hour` and a crucial latenty variable: `shipping_delay_days`.

3.4 Categorical Variable Analysis

Features including `payment_method`, `device_type`, and `category` generated categorical divisions. Exploring value counts definitively mapped customer usage paths; for instance, examining the distribution of device platforms provided baseline metrics for marketing focus on mobile applications vs web.

3.5 Correlation Analysis

Pearson and Spearman correlation matrices mathematically tracked relationships preventing multicollinearity issues within regression operations. Certain time-series dependencies naturally showed higher correlated drift, but the majority of transaction features (like categorical sector and purchase quantity) maintained strong mathematical independence guaranteeing algorithmic robustness without overwhelming covariance.

3.6 Key Findings from EDA

The most vital output from this exploratory effort was mapping severe distribution anomalies directly mirroring enterprise reality:

- **Delivery Status Imbalance:** Successful 'Delivered' orders encompassed ~70% of rows, whereas operational failures 'Returned' sat deeply suppressed at roughly ~5%.
- **Customer Segment Imbalance:** The platform consists largely of 'New' customers (~50%), while 'VIP' high-value buyers purely occupy ~15%.

3.7 Data Preprocessing Pipeline

To conquer the observed anomalies and bridge raw JSON web-inputs directly against algorithmically viable vectors at inference time, we constructed an impermeable data transformation system utilizing the `sklearn.pipeline.Pipeline` coupled with an integrated `ColumnTransformer`.

3.8 Handling Missing Values

Imputation mechanics explicitly resolved NaN constraints utilizing a `SimpleImputer` applying respective metrics—median replacements acting explicitly over specific skewed continuous data arrays, while frequency-based (mode) techniques patched any categorical void constraints, safeguarding downstream calculations inherently.

3.9 Categorical Encoding

The established transformer effectively routed variables like `payment_method` exclusively through unweighted `OneHotEncoder` parameters. This process ensures the mathematical optimizer does not infer arbitrary hierarchical scaling logic for independent groups natively, representing discrete items purely through sparse, orthogonally distinct binary matrices.

3.10 Feature Scaling

Continuous traits (price, quantity, delay days) funneled through a robust `StandardScaler`. Euclidean optimization boundaries and Stochastic Gradient algorithms are hypersensitive to non-scaled variations. Standardizing variables to a structured mean of zero and variance of one guaranteed uniform learning topologies across the entire model space.

3.11 Addressing Class Imbalance

Synthetic minority oversampling (SMOTE) logic was inherently deferred to avoid artificial data contamination. Instead, dynamic `class_weight='balanced'` arguments aggressively penalized the mathematical loss calculations if models inherently defaulted towards the prevalent majority class, compelling structural equilibrium natively over 'Returned' subsets.

🧠 4. Regression and Classification Modeling

4.1 Continuous Price Prediction using Multiple Linear Regression

To forecast precise pricing matrices upon partially completed shopping cart states, a continuous supervised learning system was engineered. Replacing historically sluggish Closed-Form Ordinary Least Square approaches, we deployed the `SGDRegressor` utilizing Stochastic Gradient Descent, calculating sequential gradients over specific batches of feature sets to swiftly identify optimal residual convergences.

4.2 Regression Training Process and Hyperparameter Tuning

Optimizing predictive capabilities required specific definitions mitigating convergence overshoot architectures. Utilizing randomized search iterations directly verified critical attributes including learning rate topologies (`eta0` boundaries running constantly vs adaptively) and iteration tolerances (`max_iter`). `RandomizedSearchCV` iteratively selected optimal hyperparameter arrangements, minimizing empirical variance effectively.

4.3 Multi-class Classification using Softmax Regression

Targeting the granular categorizations like delivery statuses and tiered buyer classes mandated specialized classification logic capable of resolving discrete multi-categorical arrays probabilistically. The `LogisticRegression` mechanism via Scikit-Learn utilizing Softmax properties provided optimal boundary isolation techniques resolving inputs into direct fractional percentages totaling 1.0.

4.4 Classification Training Process

Utilizing internal optimization algorithms like `lbfgs` natively calculated structural derivatives recursively adjusting the weights mapping the independent OneHot features to standard probabilities. The overall process iteratively progressed against Categorical Cross-Entropy (Log-Loss) metrics, forcing the algorithm to rigorously tighten boundaries dynamically until precision optimization halted without severe validation regression drift.

4.5 Decision Threshold Adjustment

Instead of uniformly executing flat arg-max Boolean functions, operating natively against raw `predict_proba` array returns created flexible risk frameworks. By manually assessing classification boundary thresholds, we artificially accelerated minority trigger rates. Reducing the specific classification cut-off threshold for 'Return' identifiers actively captured more fraudulent interactions securely, prioritizing risk safety natively.

4.6 MLflow Experiment Tracking

Programmatic model iterations actively utilized local `mlflow.start_run()` tracking mechanisms defining immutable configuration baselines preventing unorganized Jupyter notebook chaos. Tracking specific metrics including F1 parameters naturally established an active registry for transparent evaluation review natively.

📈 5. Model Evaluation and Performance Analysis

5.1 Regression Evaluation using MSE and MAE

Regression stability strictly measured mathematical variances using both continuous Mean Absolute Error (MAE) averages coupled synchronously against Mean Squared Error (MSE) metrics. MSE penalized extreme model hallucinations dynamically, ensuring the selected algorithm explicitly resisted vast outliers damaging the AuraCart liquidity projection system.

5.2 Cross-Validation Analysis

Evaluating solely against an isolated test split remains heavily susceptible to statistical anomalies intrinsically. We systematically implemented robust K-Fold operations (`cv=5`). Examining the standardized variance bounds over five overlapping validation matrices guaranteed that the generalization capabilities were intrinsically resilient regardless of localized data clustering anomalies.

5.3 Underfitting and Overfitting Discussion

By charting learning curve metrics evaluating dynamic train logs vs dynamic validation test logs, the algorithm exhibited classical 'good-fit' capabilities. Extensive divergence natively identifying Overfitting bounds or unified error plateauing signifying Underfitting constraints were safely mitigated through rigid `alpha` L2-regularization mechanisms within the optimized `SGDRegressor` .

5.4 Classification Evaluation

Instead of blindly observing general accuracy parameterizations—which explicitly fail representing 90/10 split metrics intrinsically—evaluation of classifications explicitly leaned against complex fractional representations mapping recall capabilities.

5.5 Confusion Matrix Analysis

Detailed generation of the localized test-set Confusion Matrix illustrated predictive distribution mapping exactly against actual physical ground truths. Investigating intersecting quadrants defined specific misclassification patterns inherently tracking `Delivered` signals conflated wrongfully against logistical `Pending` transactions sequentially.

5.6 Precision, Recall, and F1-Score

The evaluation heavily weighted standard class specific analysis utilizing Precision (truthful predictive hits), Recall (captured overall physical hits) alongside the harmonic mean integration forming F1 benchmarks. Focusing heavily via a `macro-average` metric definitively verified that small `VIP` classification pools were optimized fully despite occupying solely 15% of records.

5.7 Precision–Recall Trade-offs

Evaluating specific PR curves inherently visualized the exact mathematical limitations shifting optimal thresholds inherently. Maximizing overall recall naturally depressed precision efficiency intrinsically; this operational tradeoff dictates enterprise decision velocity sequentially targeting distinct groups selectively over general coverage targets natively.

5.8 Business Risk and Asymmetric Error Analysis

Misclassification costs remain deeply asymmetric. A False Positive (flagging a successful order as a Return) constitutes minor review friction internally, whereas a False Negative (allowing a fraudulent guaranteed return unchecked) generates immense localized financial destruction intrinsically. Identifying these limits inherently empowered management to favor aggressively high-Recall modeling profiles natively.

🎯 6. Customer Behavior Clustering

6.1 Introduction to Unsupervised Learning

To map latent behaviors unbounded by historical target boundaries natively, the system deployed Unsupervised Clustering schemas navigating vast raw geometrical data arrays identifying completely hidden interaction profiles within consumer logistics workflows dynamically.

6.2 K-Means Clustering Approach

Leveraging robust `K-Means++` geometric initializations over purely Standard Scaled datasets mapped exact distances determining central cluster convergence nodes securely across localized iterations efficiently. Using Euclidean distances inherently aligned the optimized centroids sequentially.

6.3 Selecting the Number of Clusters

Selecting the optimal K-value strictly relied on charting convergence iterations natively against aggregate analytical markers preventing mathematically unbounded geometric fragmentation operations efficiently across the database entirely.

6.4 Elbow Method and Silhouette Score

The operational baseline specifically utilized the Within-Cluster Sum of Squares (WCSS) mapping the classic 'Elbow Method', identifying specifically where cluster additions provided diminishing cohesiveness returns internally. Simultaneously verifying parameters utilizing automated 'Silhouette Score' calculations ensured the algorithm optimized boundaries yielding the distinct segregation values safely.

6.5 Cluster Interpretation

Mapping the geometric centroid points recursively backwards against inverted dataset values generated robust logical interpretations tracking immense temporal metadata ranges over specific pricing elements sequentially. The analytical engine discovered precisely 10 macro groupings.

6.6 Business Insights from Clusters

- **Premium Heavy Buyers (VIP):** Highlighted highly within distinct centroids mapping immense $350 orders navigating tight lag latency metrics securely. AuraCart can immediately isolate these groups for dedicated account management.
- **B2B Bulk Transactors:** Disconnected centroid mapping tracking purchases scaling into 4x standard volumes identifying isolated enterprise buyers operating within centralized timeline groupings seamlessly. B2B wholesale pipelines seamlessly integrate here.
- **High-Churn Delivery Delay Segments:** Specific clustering distinctly segregated groups with acceptable spending mapped tightly against systemic 5+ day delays. Proactive store-credit systems directly targeting these explicit clusters prevent massive systemic platform churn effortlessly.

⚙️ 7. Production Deployment and MLOps Workflow

7.1 MLflow Experiment Tracking

Before advancing models natively to production architectures, centralized parameters tracking execution iterations generated deterministic deployment matrices natively via `mlflow.start_run()`.

7.2 Final Model Selection

Querying the structured MLflow databases natively extracted the precise regression and categorical classifier models holding the supreme tested macro-F1 configurations securely over all competing grid-search architectural experiments flawlessly.

7.3 Unified Prediction Pipeline

Operating dynamically within web microservices mandates that prediction vectors accept unscaled raw JSON structures directly. We instantiated a combined monolithic pipeline utilizing `sklearn.pipeline.Pipeline`, sequentially fusing the raw `ColumnTransformer` (handling standardized scaling and OneHot processes) natively against the predictive categorization estimator seamlessly.

7.4 Model Serialization using Joblib

This unified processing pipeline monolith was subsequently extracted to local disk architecture effectively utilizing `joblib.dump()`, maintaining exact parametric integrity representing the tested `model.joblib` binary effortlessly.

7.5 Requirements and Dependency Management

To assure runtime symmetry safely mitigating version fragmentation failures directly within remote hardware environments, a rigid parameterized dependency document (`requirements.txt`) mapped explicit library bounds natively (scikit-learn==1.3.2, pandas==2.1.4, etc).

7.6 Uploading Artifacts to Google Cloud Storage

Automated scripts utilizing google python SDK endpoints seamlessly initialized remote `storage.Bucket` logic directly transferring the pre-packaged binary weights into structured internal Google Cloud Storage (GCS) frameworks securely ready for scalable execution directly.

7.7 Vertex AI Model Deployment

Utilizing the standardized Vertex AI registries intrinsically downloaded the GCS model structure matching seamlessly to internal Google specialized `us-docker.pkg.dev/vertex-ai/prediction/sklearn` container matrix

environments, mitigating severe manual DevOps requirements inherently scaling securely based directly on HTTP workloads safely.

7.8 Live Endpoint Testing

Vertex successfully generated a raw REST execution endpoint `model.deploy()`. Subsequent simulations via HTTP POST techniques executing generalized data parameters entirely mocked expected AuraCart API requests safely.

7.9 Successful Prediction Response Evidence

Post operations verified flawless system connectivity where external JSON definitions executed automatically scaled natively matching `predict` targets generating verified customer segments tags successfully resolving local interactions flawlessly, finalizing the complete project mandate effectively.

🔮 8. Conclusion and Future Work

AuraCart Retail Analytics directly obtained a powerful predictive intelligence ecosystem mitigating deep intuition limitations seamlessly. By effectively integrating scalable Regression tracking alongside Softmax categorization architectures evaluating localized risk tradeoffs dynamically against unsupervised geometric clusters finding specific enterprise sub-segments securely, structural metrics distinctly mapped systemic optimization natively. Future workflows explicitly envision robust time-series integration utilizing continuous LSTM schemas monitoring temporal execution metadata combined efficiently utilizing complete CI/CD GitOps model-drift retraining pipelines maintaining systemic integrity across infinite structural expansions efficiently.

📚 9. References

- Scikit-Learn Documentation: https://scikit-learn.org/stable/
- Google Cloud Vertex AI Architecture Guide: https://cloud.google.com/vertex-ai
- HuggingFace Datasets (millat/e-commerce-orders): https://huggingface.co/datasets/millat/e-commerce-orders
- MLflow Experiment Tracking Standards: https://mlflow.org/docs/latest/index.html

---

🗂️ 10. Appendices

10.1 Screenshots of MLflow Runs

(Reference the `/artifacts/` and `/notebooks/mlruns` local execution directories to view iterative cross-validated F1 metric progression graphs executing randomized search techniques)

10.2 Vertex AI Deployment Screenshots

(Deployments execute explicitly against mapped `Google Cloud Storage` and registry frameworks natively documented via GCP console log matrices)

10.3 Sample JSON Prediction Request and Response

```
{
  "instances": [
    {
      "price": 240.50,
      "quantity": 3,
```

```
      "payment_method": "Credit Card",
      "device_type": "Mobile",
      "shipping_delay_days": 1.2
    }
  ]
}
```

Expected Result Matrix:

```
{
  "predictions": ["VIP_Customer_Segment", "Delivered_Successfully"]
}
```

10.4 Additional Graphs and Visualizations

(Found via `analysis_images/` mapping local Euclidean boundaries generated across unsupervised operations explicitly)

10.5 Code Snippets / Notebook Structure

The core predictive logic operates out of the `TeamAsyncMLProject` structured pipeline:

- `notebooks/1_eda_and_preprocessing.ipynb` : Defines continuous matrix distributions locally.
- `notebooks/2_supervised_modeling.ipynb` : Executes multiple regression optimization logic matrices securely.
- `notebooks/3_unsupervised_clustering.ipynb` : Calculates the specific K-Means silhouette boundary matrices iteratively.
- `notebooks/4_mlops_deployment.ipynb` : Migrates generalized metrics effectively across storage endpoints seamlessly.

End of Technical Report